

# **SEMESTER TRAINING (BTCS 801-18)**

## **REPORT**

**on**

**Buildr: AI-Powered Website Builder Using Next.js and OpenAI**

*Submitted in partial fulfilment of requirements for the award of the degree*

***Bachelor of Technology***

***In***

***Information Technology***

**To**

**IKG Punjab Technical University, Jalandhar**

**SUBMITTED BY:**

Name: Rajat Khurana

Roll No.: 2237440

Semester: 8<sup>th</sup>

Batch: 2022-26

**Under the Guidance**

**Mr. Amitabh Sharma and Mr. Kamaljit Singh**

**Assistant Professor**



**DEPARTMENT OF INFORMATION TECHNOLOGY**

**Chandigarh Engineering College- CGC**

**Landran, Mohali, Punjab – 140307**

***May 2026***

# DECLARATION

I hereby declare that the Project Report entitled (Buildr: AI-Powered Website Builder Using Next.js and OpenAI) is an authentic record of my own work as requirements of 8<sup>th</sup> semester academics as Semester Training BTCS 801-18 during the period from January 2026 to May 2026 for the award of degree of B. Tech. Information Technology, Chandigarh Engineering College-CGC Landran, Mohali under the guidance of Mr. Amitabh Sharma and Mr. Kamaljit Singh

**NAME: Rajat Khurana**

**Roll No: 2237440**

**Date:**

**Signatures**

**Examined by:**

1.

2.

3.

4.

**Head of Department**

**(Signature and Seal)**

## **Acknowledgement**

I take this opportunity to express my sincere gratitude to the Director- Principal **Dr. Rajdeep Singh**, Chandigarh Engineering College-CGC Landran, Mohali for providing this opportunity to carry out the present work.

I am highly grateful to **Dr. Amanpreet Kaur** HOD IT, Chandigarh Engineering College-CGC Landran, Mohali for her invaluable support. I would like to express my gratitude to other faculty members of Information Technology department of CEC-CGC, Landran, Mohali for providing academic input, guidance & Encouragement throughout the training period.

The help rendered by **Mr. Amitabh Sharma and Mr. Kamaljit Singh** Supervisor for training work is greatly acknowledged. Finally, I express my indebtedness to all who have directly or indirectly contributed to the successful completion of my semester training.

## **About Institute**

## **Institute Overview**

*Chandigarh Engineering College* is a premier educational institution dedicated to providing quality technical education to students across various disciplines. Established in 2001, the institute has consistently strived to foster academic excellence, innovation, and research among its students.

## **Vision**

To become a leading institution of technical education that nurtures talented professionals equipped with knowledge, skills, and values to contribute meaningfully to society and industry.

## **Mission**

- To provide quality education through innovative teaching methodologies
- To encourage research and development among students and faculty
- To prepare students for the challenges of the ever evolving technology landscape
- To foster a culture of creativity, critical thinking, and problem solving

## **Department of Information Technology**

The Department of Information Technology at Chandigarh Engineering College offers a comprehensive B.Tech program designed to equip students with strong theoretical foundations and practical skills in areas including:

- Software Development
- Database Management
- Artificial Intelligence and Machine Learning
- Web Technologies
- Computer Networks

The department is equipped with modern laboratories, experienced faculty, and industry-oriented curriculum that prepares students for successful careers in the technology sector.

## Table of Contents

| Chapter | Title                                                | Page No. |
|---------|------------------------------------------------------|----------|
| -       | Acknowledgement                                      | i        |
| -       | About Institute                                      | ii       |
| -       | Table of Contents                                    | iii      |
| -       | List Of Figures                                      | vi       |
| -       | List of Tables                                       | vii      |
| -       | Abbreviations and Nomenclature                       | viii     |
| -       | Abstract of the Project                              | ix       |
| 1       | Introduction                                         | 1        |
| 1.1     | Brief Overview of Work                               | 2        |
| 1.2     | Objective                                            | 4        |
| 1.3     | Scope                                                | 5        |
| 1.4     | Project Modules                                      | 6        |
| 1.5     | Project Requirements                                 | 8        |
| 2       | System Analysis                                      | 10       |
| 2.1     | Literature Review                                    | 11       |
| 2.2     | Project Feasibility Study                            | 14       |
| 2.4     | Project Timeline Chart                               | 16       |
| 2.5     | Detailed Module Description with all Functionalities | 17       |
| 3       | System Design                                        | 21       |
| 3.1     | Use Case Diagrams                                    | 22       |
| 3.2     | Data Flow Diagrams                                   | 24       |
| 3.3     | Class Diagram                                        | 28       |
| 3.4     | Entity Relationship Diagram                          | 30       |
| 3.5     | Activity Diagram                                     | 32       |
| 3.6     | Data Dictionary                                      | 34       |
| 4       | Software Tools                                       | 38       |
| 4.12    | Environment Variables and Configuration              | 56       |
| 4.13    | Folder Structure                                     | 57       |
| 5       | Implementation and Testing                           | 59       |
| 5.1     | User Interface and Snapshots                         | 61       |
| 5.2     | Test Cases and Result                                | 68       |
| 5.3     | Important Implementation Files                       | 70       |
| 5.4     | Database Implementation                              | 71       |

|     |                                      |    |
|-----|--------------------------------------|----|
| 5.5 | Background Generation Implementation | 72 |
| 5.6 | Deployment and Execution             | 73 |
| 6   | Conclusion and Future Scope          | 75 |
| -   | References                           | 78 |

## List Of Figures

| Figure No. | Title                                 | Page No. |
|------------|---------------------------------------|----------|
| 3.1        | Use Case Diagram of Buildr            | 23       |
| 3.2        | Context-Level DFD of Buildr           | 25       |
| 3.3        | Level 1 DFD of Buildr                 | 27       |
| 3.4        | Class Diagram of Buildr               | 29       |
| 3.5        | Entity Relationship Diagram of Buildr | 31       |
| 3.6        | Activity Diagram of Buildr            | 33       |
| 5.1        | Home Page of Buildr                   | 58       |
| 5.2        | Project Workspace of Buildr           | 59       |
| 5.3        | Generated Website Preview             | 60       |
| 5.4        | Generated Code Explorer               | 61       |
| 5.5        | Authentication Screens                | 62       |
| 5.6        | Database Schema or Prisma Models      | 63       |

## List of Tables

| Table No. | Title                              | Page No. |
|-----------|------------------------------------|----------|
| 1.1       | Hardware Requirements              | 8        |
| 1.2       | Software Requirements              | 9        |
| 2.1       | Feasibility Study                  | 15       |
| 2.2       | Project Timeline                   | 16       |
| 3.1       | Data Dictionary for Project Table  | 35       |
| 3.2       | Data Dictionary for Message Table  | 36       |
| 3.3       | Data Dictionary for Fragment Table | 37       |
| 4.1       | Software Tools Used in Buildr      | 55       |
| 5.1       | Functional Test Cases              | 66       |

## Abbreviations and Nomenclature

| Abbreviation | Full Form                          |
|--------------|------------------------------------|
| AI           | Artificial Intelligence            |
| API          | Application Programming Interface  |
| CSS          | Cascading Style Sheets             |
| DFD          | Data Flow Diagram                  |
| ER           | Entity Relationship                |
| HMR          | Hot Module Replacement             |
| JSON         | JavaScript Object Notation         |
| ORM          | Object Relational Mapping          |
| PDF          | Portable Document Format           |
| SRS          | Software Requirement Specification |
| trPC         | TypeScript Remote Procedure Call   |
| UI           | User Interface                     |
| URL          | Uniform Resource Locator           |

## **Abstract of the Project**

Buildr is an AI-powered web application builder that converts natural language prompts into complete website prototypes. The system allows a user to describe a website requirement, creates a project automatically, sends the work to an AI coding workflow, runs generated code inside an isolated sandbox and displays both a live preview and generated source code in the browser.

The project has been developed using Next.js, React, TypeScript, Tailwind CSS, Shadcn/UI, tRPC, Prisma, PostgreSQL, Clerk, Inngest, OpenAI and E2B. These tools together provide a modern full-stack architecture with authentication, project management, type-safe APIs, database persistence, background processing, usage tracking and safe code execution.

This report follows the prescribed semester training format and the structure of the provided 8th semester sample report. It contains the introduction, system analysis, system design, software tools, implementation details, testing, conclusion, future scope and references. The report also includes hardware and software requirements, feasibility study, diagrams, data dictionary and test cases related to the Buildr project.

Buildr demonstrates how artificial intelligence can support rapid software prototyping. It is useful for students, developers, entrepreneurs and non-technical users who need to convert ideas into usable web interfaces quickly while still being able to inspect the generated code. The project shows that AI becomes more practical when it is connected with a real project workflow, persistent data and a safe execution environment.

# Chapter 1 : Introduction

Buildr is a full-stack AI website builder designed to reduce the effort required to create a working web prototype. A user enters a prompt such as a portfolio website, business landing page, dashboard or product page, and the system converts that requirement into generated website files and a running preview.

The project is based on the idea that modern artificial intelligence can act as a coding assistant when it is connected to a real development environment. Buildr provides that environment through a Next.js application, a PostgreSQL database, an Inngest background workflow and an E2B sandbox for safe execution.

Unlike a simple chatbot, Buildr stores projects and messages, generates code in an isolated workspace and displays the result through a preview and file explorer. This makes the project useful not only for output generation but also for learning and reviewing the generated implementation.

## **1.1 Brief Overview of Work**

The work completed in this project includes design and implementation of an AI-powered project generation system. The application begins with a prompt form on the home page. After authentication, the user can create projects, submit messages, view generated results and continue improving the website through follow-up prompts.

The backend records every project and message using Prisma and PostgreSQL. The generation task is sent to Inngest because website generation may take more time than a normal web request. The Inngest function creates a sandbox, runs an AI coding agent, saves generated files and stores the final result in the database.

The user interface is divided into a message area, a live demo area and a code viewing area. This layout helps the user understand both the final result and the source files produced by the AI agent. The generated preview shows whether the prompt was understood correctly, while the file explorer makes the generated structure visible.

The project also includes credit tracking so that every generation consumes a usage point. This is important because AI models and sandbox environments consume real resources. The credit system helps control usage while still keeping the application simple for users.

## 1.2 Objective

The main objective of Buildr is to provide a web platform that can generate a functional website from a natural language prompt. The project also aims to make AI generation practical by connecting it with authentication, database storage, background processing and live preview.

The system is intended to support quick prototyping and learning. Users can experiment with ideas, observe the generated output and inspect the code structure without setting up a project manually. This makes Buildr useful in academic demonstrations, project planning and early product validation.

- To accept website requirements in natural language.
- To create and maintain user-specific projects.
- To store prompt and response history for every project.
- To run AI code generation in the background.
- To execute generated code in an isolated sandbox.
- To display live preview and generated source code.
- To maintain usage control for fair resource consumption.

### **1.3 Scope**

The scope of the project includes user authentication, prompt submission, project creation, message history, background AI generation, sandbox execution, result storage and preview display. Buildr is suitable for generating early-stage web prototypes and small application interfaces.

The project does not claim to replace complete software engineering practices. Generated code should be reviewed before production use. However, it significantly reduces the time needed to move from an idea to a demonstrable web interface.

The current scope is focused on web application generation using a Next.js sandbox template. Future versions may include multi-framework support, direct deployment, design editing, version history and collaborative project workspaces.

## **1.4 Project Modules**

The Buildr project is divided into multiple modules so that each responsibility is clear. The home module manages project creation, the project module handles project retrieval, the message module stores conversations, the Inngest module performs AI generation and the UI components render preview and code.

This modular approach makes the application easier to maintain. If the preview interface changes, the background generation function does not need to be rewritten. Similarly, if the AI prompt is improved, the database design and authentication flow can remain the same.

### **1.4.1 Authentication**

Authentication is managed through Clerk. Users must sign in before creating or opening protected projects. The authenticated user identifier is stored with each project and checked in protected server procedures.

### **1.4.2 Prompt Processing**

Prompt processing starts when the user submits a text requirement. The input is validated, usage credits are consumed and a project or message record is created. The same prompt is then sent to the background generation function through an Inngest event.

### **1.4.3 Project and Message Management**

Project management stores project names, owners and timestamps. Message management stores both user prompts and assistant responses. These records allow the user to return to previous work and continue improving a generated website.

### **1.4.4 Preview and Code Explorer**

The preview module displays the generated website through an iframe using the sandbox URL. The code explorer reads stored fragment files and presents them in a file tree with syntax-highlighted content.

## 1.5 Project Requirements

The project requirements include hardware and software resources needed for development, execution and usage. Since Buildr is a web application, client-side requirements are modest, while the backend requires API keys and service configuration.

The development environment requires Node.js, npm and a code editor. The production environment requires hosting for the Next.js application, a PostgreSQL database, Clerk configuration, Inngest endpoint configuration, OpenAI access and E2B sandbox support.

**Table 1.1: Hardware Requirements**

| Component | Minimum Requirement | Recommended Requirement       |
|-----------|---------------------|-------------------------------|
| Processor | Dual-core processor | Quad-core processor or higher |
| RAM       | 4 GB                | 8 GB or higher                |
| Storage   | 1 GB free space     | 5 GB free space               |
| Network   | Internet connection | Stable broadband connection   |
| Client    | Modern web browser  | Chrome, Edge or Firefox       |

### 1.5.1 Software

The software requirements include the tools and services needed to build and run the application. Node.js and npm are used for development, while the application framework and backend services handle user-facing and server-side behaviour.

**Table 1.2: Software Requirements**

| Software                   | Purpose                             |
|----------------------------|-------------------------------------|
| Next.js and React          | Application framework and interface |
| TypeScript                 | Type-safe development               |
| Tailwind CSS and Shadcn/UI | Styling and reusable components     |
| tRPC and React Query       | Type-safe API communication         |
| Prisma and PostgreSQL      | Database schema and storage         |
| Clerk                      | Authentication                      |
| Inngest                    | Background workflow                 |
| OpenAI                     | AI generation                       |
| E2B                        | Sandbox execution                   |



## **Chapter 2 : System Analysis**

System analysis studies the existing problem, evaluates feasibility and defines how Buildr should behave. The project solves the problem of slow manual prototyping by using AI agents and sandbox execution.

The analysis phase identified that the system must be easy for users but technically reliable behind the scenes. The user should only need to provide a prompt, while the application should manage authentication, storage, background execution and preview generation automatically.

## 2.1 Literature Review

Traditional website development requires knowledge of HTML, CSS, JavaScript, frameworks, component libraries, state management, database access and deployment. Even a simple prototype may take significant time if created manually.

Recent developments in large language models have shown that AI can generate code from natural language. However, raw code generation is not enough. A practical system must validate input, run the generated application, store the output and show it to the user. Buildr combines these requirements into one integrated platform.

Modern full-stack frameworks such as Next.js help developers build both frontend and backend parts in one codebase. Tools such as tRPC, Prisma and Inngest make the architecture more reliable by providing typed APIs, database access and background execution.

AI-assisted development tools are most useful when they support iteration. Buildr follows this idea by storing previous messages and allowing follow-up prompts. The system can use recent project context so that users can refine the generated website instead of starting from zero every time.

## **Importance of AI Website Builders**

AI website builders are important because they reduce the distance between idea and implementation. Students can use them to learn project structure, designers can test layouts quickly and entrepreneurs can validate product ideas before investing in complete development.

Buildr is also useful as an educational tool because it exposes generated files instead of hiding the code. Users can compare their prompt with the generated output and understand how components are organised.

For semester training, this project demonstrates many important software engineering concepts, including modular design, database modelling, API routing, authentication, background jobs and external service integration.

## 2.2 Project Feasibility Study

The feasibility study determines whether Buildr is practical from technical, economic and operational perspectives. The selected technologies are widely used and can be integrated through APIs and SDKs.

**Table 2.1: Feasibility Study**

| Type                    | Description                                                            |
|-------------------------|------------------------------------------------------------------------|
| Technical Feasibility   | Achievable with Next.js, Prisma, Inngest, OpenAI and E2B               |
| Economic Feasibility    | Uses open-source tools with paid external services only where required |
| Operational Feasibility | Prompt-based workflow is simple for end users                          |
| Schedule Feasibility    | Modular design allows staged development and testing                   |

### 2.2.1 Technical Feasibility

The project is technically feasible because the required functionality can be implemented using existing libraries and cloud services. Next.js supports full-stack routes, Clerk supports authentication, Prisma supports database access, Inngest supports background jobs and E2B supports isolated execution.

### 2.2.2 Economical Feasibility

The project is economically feasible because most development tools are open-source. Costs are mainly associated with AI model usage, sandbox execution and hosting. Usage tracking can control resource consumption.

### 2.2.3 Operational Feasibility

The system is operationally feasible because users interact with a simple chat-like workflow. They do not need to understand the internal AI or sandbox process. They only need to enter requirements and review the generated result.

## 2.4 Project Timeline Chart

The project was completed in phases including requirement study, design, implementation, testing and report preparation.

**Table 2.2: Project Timeline**

| Phase  | Work Completed                           |
|--------|------------------------------------------|
| Week 1 | Requirement collection and project study |
| Week 2 | Technology selection and database design |
| Week 3 | Authentication and project module        |
| Week 4 | Message flow and tRPC procedures         |
| Week 5 | Inngest agent and sandbox integration    |
| Week 6 | Preview, code explorer and testing       |
| Week 7 | Documentation and final report           |

## **2.5 Detailed Module Description with all Functionalities**

The home module provides the first interaction point. It accepts a prompt and shows a list of existing projects. The project module verifies ownership and loads project details. The message module stores user and assistant communication.

The background generation module is responsible for sandbox creation, previous message loading, AI agent execution, title generation, response generation and result saving. The usage module prevents unlimited generation by consuming credits.

The UI modules use reusable components such as buttons, tabs, cards, file explorers and code viewers. This keeps the interface consistent and reduces duplication across the application.

### **2.5.1 Registration and Login**

Registration and login are provided by Clerk. After successful login, the user receives an authenticated session. The application uses the session to allow access to protected routes and server procedures.

### **2.5.2 Project Creation**

Project creation happens when a user submits the first prompt. The system validates the prompt, creates a project name, stores the message and starts the generation workflow.

### **2.5.3 Search and Project Listing**

The project listing feature displays projects belonging to the logged-in user. Projects are ordered by update time so that recent work appears first.

### **2.5.4 Manage Account and Usage**

User account management is handled by Clerk. Usage management is handled by the credit system, which limits the number of generations available to a user.



## **Chapter 3 : System Design**

System design describes the structural view of Buildr. It includes use case design, data flow diagrams, class relationships, entity relationship diagram, activity diagram and data dictionary.

The design focuses on separating responsibilities. The browser interface handles user interaction, tRPC handles API communication, Prisma handles database access, Inngest handles background execution and E2B handles sandboxed generated code.

### 3.1 Use Case Diagrams

The use case diagram identifies the main actions performed by the user. The user can sign in, create a project, submit prompts, view messages, preview generated websites and inspect code.

Actor: User

Use Cases: Login, Create Project, Submit Prompt, View Preview, Inspect Code, Continue Iteration

External Services: Clerk, Inngest, OpenAI, E2B

**Figure 3.1: Use Case Diagram of Buildr**

## 3.2 Data Flow Diagrams

Data flow diagrams explain how information moves through the system. The prompt flows from the UI to the tRPC API, then to the database and background generation function.

User -> Buildr Web UI -> tRPC API  
tRPC API -> PostgreSQL Database  
tRPC API -> Inngest Event  
Inngest -> OpenAI Agent + E2B Sandbox  
Sandbox Result -> Database Fragment -> User Preview

**Figure 3.2: Context-Level DFD of Buildr**

### 3.2.1 Context-Level (Level 0)

At context level, Buildr is represented as a single system interacting with the user and external services. The user provides prompts and receives generated websites. External services provide identity, AI generation and sandbox execution.

### 3.3.2 Level 1 DFD

At level 1, the system is divided into authentication, project management, message management, background generation and preview display. Each process communicates with the database or an external service as required.

1. Authentication validates user
2. Project module stores project
3. Message module stores prompt
4. Inngest runs AI generation
5. Fragment module stores URL and files
6. UI renders preview and code

**Figure 3.3: Level 1 DFD of Buildr**

### 3.3 Class Diagram

The class diagram represents the logical structure of the system. Project, Message, Fragment and Usage are the most important data classes. UI components and server procedures operate on these classes through Prisma.

Project: id, name, userId, createdAt, updatedAt

Message: id, content, role, type, projectId

Fragment: id, messageId, sandboxUrl, title, files

Usage: key, points, expire

**Figure 3.4: Class Diagram of Buildr**

### 3.4 Entity Relationship Diagram

The ER diagram shows that one project can have many messages and one message can have one fragment. Usage is a separate table used for credit tracking.

Project 1 ---- Many Message  
Message 1 ---- Zero or One Fragment  
Usage stores credit information independently

**Figure 3.5: Entity Relationship Diagram of Buildr**

### 3.5 Activity Diagram

The activity diagram describes the flow from user login to generated website preview. It also includes the decision point where generation may succeed or fail.

Start -> Login -> Enter Prompt -> Create Project -> Send Event  
Create Sandbox -> Run AI Agent -> Files Generated?  
Yes -> Save Fragment -> Display Preview  
No -> Save Error Message -> Display Error

**Figure 3.6: Activity Diagram of Buildr**

## 3.6 Data Dictionary

The data dictionary describes the main database fields used by Buildr. These tables are defined in the Prisma schema.

**Table 3.1: Data Dictionary for Project Table**

| Field     | Type     | Description               |
|-----------|----------|---------------------------|
| id        | String   | Unique project identifier |
| name      | String   | Generated project name    |
| userId    | String   | Owner user identifier     |
| createdAt | DateTime | Creation timestamp        |
| updatedAt | DateTime | Last update timestamp     |

### 3.6.1 Message and Fragment Tables

Messages store conversation data and fragments store generated output. These two tables connect the user-facing chat with the generated website result.

**Table 3.2: Data Dictionary for Message Table**

| Field     | Type   | Description                  |
|-----------|--------|------------------------------|
| id        | String | Unique message identifier    |
| content   | String | Prompt or assistant response |
| role      | Enum   | USER or ASSISTANT            |
| type      | Enum   | RESULT or ERROR              |
| projectId | String | Related project identifier   |

### 3.6.2 Fragment Table

A fragment is created when AI generation succeeds. It contains the sandbox URL, generated title and the file map used by the code explorer.

**Table 3.3: Data Dictionary for Fragment Table**

| Field      | Type   | Description                |
|------------|--------|----------------------------|
| id         | String | Unique fragment identifier |
| messageId  | String | Related assistant message  |
| sandboxUrl | String | Live preview URL           |
| title      | String | Generated fragment title   |
| files      | Json   | Generated source files     |



## **Chapter 4 : Software Tools**

This chapter explains the software tools and technologies used in Buildr. The project uses a modern TypeScript-based full-stack environment with AI and sandbox services.

The software stack was selected so that the application can support a complete workflow from prompt submission to generated website preview. Each tool has a specific role in the architecture and helps keep the project maintainable.

## **4.1 Overview of Next.js Technology**

Next.js is the main framework used in Buildr. It provides routing, layouts, API routes and server-side capabilities in one application. The App Router structure keeps pages and API endpoints organised.

In Buildr, Next.js is used for the home page, authentication pages, project pages and API route handlers. It gives the project a production-ready foundation and works naturally with React, TypeScript, Clerk and deployment platforms.

### **4.1.1 Features of Next.js**

Important features include file-based routing, server components, client components, hot reload, API routes and production optimisation. These features make Next.js suitable for a full-stack AI builder platform.

The App Router also allows layouts and nested routes. This is useful for separating the home area, pricing page, sign-in pages and project workspace without creating complex routing code manually.

### **4.1.2 React and Component Based UI**

React is used to build reusable interface components such as forms, message cards, tabs, panels and file explorers. Its component model keeps the user interface maintainable.

The project page contains interactive components that need browser state, such as selected tabs, selected files, message form state and preview refresh behaviour. React hooks support these interactions clearly.

### **4.1.3 TypeScript**

TypeScript adds static typing to JavaScript. It improves reliability by checking component props, API inputs, database types and utility function signatures during development.

In Buildr, TypeScript works with Prisma and tRPC to provide end-to-end type safety. This reduces mistakes when frontend components call server procedures or when server code stores data in the database.

## **4.2 About Tailwind CSS**

Tailwind CSS is a utility-first styling framework. Buildr uses it to create responsive layouts, spacing, colours and typography quickly while maintaining a consistent design.

Tailwind is useful in this project because the UI has several panels and states. Utility classes make it easy to adjust spacing, responsive behaviour and visual emphasis directly inside components.

### **4.3 Shadcn/UI and Radix UI**

Shadcn/UI provides reusable interface components based on Radix UI primitives. These components support accessibility and allow direct customisation because the component code exists inside the project.

Buildr uses common UI elements such as buttons, inputs, tabs, tooltips, dropdowns and panels. Using a consistent component library improves the professional appearance of the application.

## 4.4 tRPC

tRPC provides type-safe communication between frontend and backend. Procedures are defined on the server and called from the client without manually writing separate REST schemas.

The projects router and messages router are important parts of Buildr. They validate input, check authentication, query the database and trigger background generation events.

## **4.5 Prisma and PostgreSQL**

Prisma is the object relational mapper used to access the PostgreSQL database. The Prisma schema defines Project, Message, Fragment and Usage models.

PostgreSQL is suitable because the project data is relational. Projects contain messages, messages can contain fragments, and usage entries track resource consumption.

## **4.6 Clerk Authentication**

Clerk manages user sign-in, sign-up and session handling. Buildr uses Clerk user identifiers to protect projects and API procedures.

Authentication is important because projects are private to each user. The backend checks that the logged-in user owns a project before loading it or accepting a follow-up prompt.

## 4.7 Inngest

Inngest is used for background job execution. The AI generation process runs as an event-driven function so that long tasks do not block the main request.

The BuildrAgent function is triggered by the BuildrAgent/run event. It performs sandbox creation, previous message loading, AI agent execution, title generation, response generation and database saving.

## **4.8 OpenAI and Agent Kit**

OpenAI models are used for code generation, response generation and title generation. Agent Kit provides a tool-based agent workflow that can read files, write files and run commands.

The main coding agent receives a structured system prompt and has tools for terminal execution, file reading and file writing. This makes the generated result closer to a real application than a plain text answer.

## **4.9 E2B Sandbox**

E2B provides an isolated code execution environment. Generated applications run inside the sandbox and expose a preview URL for the Buildr interface.

Sandboxing is important because AI-generated code should not execute directly inside the main application server. E2B separates generated code from the Buildr host application.

## **4.10 Development Tools**

The project also uses npm, ESLint, TypeScript configuration, Prisma migrations, Docker files and sandbox templates. These tools help with development, quality control and deployment preparation.

The sandbox template contains the base environment for generated Next.js applications. It allows generated projects to start from a consistent foundation.

## 4.11 Summary of Software Tools

The following table summarises the main tools used in the Buildr project and the reason each tool was selected.

**Table 4.1: Software Tools Used in Buildr**

| Tool         | Use in Project                     |
|--------------|------------------------------------|
| Next.js      | Full-stack application framework   |
| React        | Interactive user interface         |
| TypeScript   | Static typing and maintainability  |
| Tailwind CSS | Responsive styling                 |
| Shadcn/UI    | Reusable UI components             |
| tRPC         | Type-safe API communication        |
| Prisma       | Database access                    |
| PostgreSQL   | Persistent storage                 |
| Clerk        | Authentication                     |
| Inngest      | Background generation workflow     |
| OpenAI       | AI code and response generation    |
| E2B          | Sandbox execution and live preview |

## 4.12 Environment Variables and Configuration

Buildr requires several environment variables to connect the application with external services. These values are not written directly in source files because they contain sensitive information such as database URLs and API keys.

The database connection string is used by Prisma to connect with PostgreSQL. Clerk keys are used for authentication. OpenAI keys are used by the AI agents. Inngest and E2B configuration values allow background jobs and sandbox execution to work correctly.

Keeping configuration outside the source code improves security and makes deployment easier. The same codebase can run in development and production by changing environment values only.

- `DATABASE_URL` connects Prisma with PostgreSQL.
- Clerk keys enable authentication and protected routes.
- `OPENAI_API_KEY` enables AI generation.
- Inngest configuration enables background event processing.
- E2B configuration enables sandbox creation and preview URLs.

## 4.13 Folder Structure

The folder structure of Buildr follows a feature-based approach. Application routes are placed under `src/app`, reusable components under `src/components`, project features under `src/modules` and background functions under `src/inngest`.

The Prisma folder contains schema and migrations. The sandbox-templates folder contains the base template used for generated Next.js applications. This separation makes it easier to locate files during development and viva explanation.

A clean folder structure is important in a project like Buildr because it combines frontend, backend, database and AI workflows in one repository. Without clear organisation, the project would become difficult to maintain.

- `src/app` contains pages, layouts and API route handlers.
- `src/modules` contains home, project, message and usage features.
- `src/trpc` contains router setup and client/server helpers.
- `src/inngest` contains the BuildrAgent background function.
- `prisma` contains database schema and migration files.
- `sandbox-templates` contains the Next.js generation template.



## **Chapter 5 : Implementation and Testing**

Implementation converts the system design into working software. Buildr has been implemented through frontend modules, server procedures, database schema, background functions and sandbox templates.

The implementation follows a feature-based structure. UI files are placed under modules, API logic is kept in server procedures, database models are stored in the Prisma schema and the AI workflow is implemented inside the Inngest function.

## **Implementation**

The implementation starts from the home page where the user enters a prompt. The prompt is sent to the projects router, which validates the input, consumes credit, creates a project and sends a BuildrAgent/run event.

The background function creates a sandbox, loads previous messages, runs the AI coding agent and stores the generated result. The project page then shows messages, preview and code.

The project also includes route protection. A user can only load projects associated with the authenticated user identifier. This prevents unauthorised access to other users' projects.

## 5.1 User Interface and Snapshots

The user interface is designed around the builder workflow. The first screen accepts a prompt, while the project screen contains the chat history, preview and source code.

The following placeholders show exactly where screenshots should be inserted. Screenshots should be taken from the running Buildr application after signing in and creating at least one project.

**[INSERT SCREENSHOT HERE: Home page with prompt form and project list]**

*Paste the screenshot/image in this space and keep the caption below it.*

**Figure 5.1: Home Page of Buildr**

### 5.1.1 Project Workspace

The project workspace displays the conversation on the left and the generated output on the right. The right panel contains tabs for demo preview and code view.

**[INSERT SCREENSHOT HERE: Project page showing messages and right-side preview/code panel]**

*Paste the screenshot/image in this space and keep the caption below it.*

**Figure 5.2: Project Workspace of Buildr**

### 5.1.2 Generated Website Preview

The generated website preview is loaded from the sandbox URL. This gives immediate visual feedback after generation completes.

**[INSERT SCREENSHOT HERE: Demo tab with generated website loaded in iframe]**

*Paste the screenshot/image in this space and keep the caption below it.*

**Figure 5.3: Generated Website Preview**

### 5.1.3 Generated Code Explorer

The code explorer displays generated files stored in the fragment. This feature makes the system transparent and helps users learn from the generated source code.

**[INSERT SCREENSHOT HERE: Code tab showing file explorer and generated source code]**

*Paste the screenshot/image in this space and keep the caption below it.*

**Figure 5.4: Generated Code Explorer**

### 5.1.4 Authentication Screens

Authentication screens show how the user enters the protected application. These screens are useful in the report because they demonstrate that project access is user-specific.

**[INSERT SCREENSHOT HERE: Clerk sign-in or sign-up page]**

*Paste the screenshot/image in this space and keep the caption below it.*

**Figure 5.5: Authentication Screens**

### 5.1.5 Database Schema or Prisma Models

The database schema screenshot should show the Project, Message, Fragment and Usage models. It proves that the project has a defined backend data model.

**[INSERT SCREENSHOT HERE: prisma/schema.prisma showing Project, Message, Fragment and Usage models]**

*Paste the screenshot/image in this space and keep the caption below it.*

**Figure 5.6: Database Schema or Prisma Models**

## 5.2 Test Cases and Result

Testing was performed to verify that the main workflows behave correctly. The test cases cover authentication, project creation, message submission, background generation and preview display.

**Table 5.1: Functional Test Cases**

| Test Case                   | Expected Result                        | Status |
|-----------------------------|----------------------------------------|--------|
| Valid login                 | User reaches protected area            | Pass   |
| Submit valid prompt         | Project and message are created        | Pass   |
| Submit empty prompt         | Validation error is shown              | Pass   |
| Open own project            | Project details are loaded             | Pass   |
| Open another user's project | Access is denied                       | Pass   |
| Generation success          | Fragment is saved and preview is shown | Pass   |
| Generation failure          | Error message is stored                | Pass   |
| Credit exhausted            | Generation is blocked with error       | Pass   |

### Unit Testing

Unit testing focuses on individual functions and components. Input validation, utility functions, UI components and API procedure behaviour can be tested independently.

Examples include checking prompt length validation, ensuring that empty prompts are rejected and verifying that utility functions return expected values.

### Integration Testing

Integration testing verifies that modules work together. For Buildr, important integration points include tRPC with Prisma, Clerk with protected procedures, Inngeist with the database and the sandbox with generated output.

The most important integration test is the complete flow from prompt submission to saved fragment, because this confirms that frontend, backend, AI and sandbox layers communicate properly.

### Functional Testing

Functional testing verifies user-visible requirements. The application was checked for project creation, message display, preview rendering and code inspection.

### Acceptance Testing

Acceptance testing verifies whether the system satisfies the project objectives. Buildr accepts prompts, generates output and stores results, so it meets the main acceptance criteria.

## **Conversion**

Conversion refers to moving from manual website prototyping to an AI-assisted workflow. Buildr supports this by automating setup, generation and preview while still allowing human review.

## **Post Implementation Review**

After implementation, the system was reviewed for usability, maintainability and reliability. The modular structure makes future improvements easier, and the background workflow improves reliability for long-running generation tasks.

### 5.3 Important Implementation Files

Several files are important for understanding the implementation of Buildr. The project-form component accepts the first prompt. The projects server procedure creates a project and sends the Inngest event. The messages server procedure handles follow-up prompts inside an existing project.

The Inngest function is the most important backend workflow file. It creates the sandbox, loads previous messages, runs the coding agent, stores generated files and saves the final assistant response in the database.

The Prisma schema defines the database tables. The prompt file contains instructions used by the coding agent. The project view and related components display the generated output to the user.

- `src/modules/home/ui/project-form.tsx` handles the first prompt.
- `src/modules/projects/server/procedures.ts` creates and loads projects.
- `src/modules/messages/server/procedures.ts` creates and loads messages.
- `src/inngest/functions.ts` runs the AI generation workflow.
- `src/prompt.ts` stores the AI agent instructions.
- `prisma/schema.prisma` defines Project, Message, Fragment and Usage.

## 5.4 Database Implementation

The database implementation is based on Prisma models. Project records store the user-specific project information. Message records store the conversation between the user and assistant. Fragment records store generated website output linked with an assistant message.

The relationship between Project and Message is one-to-many. This allows every project to maintain complete conversation history. The relationship between Message and Fragment is one-to-one, because a successful assistant response has one generated result.

The Usage table stores credit information. This table helps the application limit generation attempts and protect resource usage. Since AI and sandbox execution can be costly, credit control is an important part of the implementation.

## 5.5 Background Generation Implementation

The background generation implementation starts when the server sends a BuildrAgent/run event. Inngest receives this event and starts the BuildrAgent function. The function creates an E2B sandbox using the configured sandbox template.

After sandbox creation, the function loads recent messages from the same project. These messages are converted into agent-compatible format and placed into the agent state. The coding agent then receives the user prompt and works with tools for terminal commands, file reading and file writing.

When generation is complete, smaller agents create a fragment title and a user-friendly response. The sandbox URL is collected and stored along with generated files. If generation fails, an error message is saved instead of a fragment.

## 5.6 Deployment and Execution

Buildr can be executed locally using `npm install` and `npm run dev` after configuring environment variables. During development, Next.js provides hot reload so that UI changes are visible quickly.

For deployment, the application requires a production PostgreSQL database, Clerk project settings, OpenAI API access, Inngest configuration and E2B sandbox support. Prisma migrations must be applied to the production database before regular use.

The application can be deployed on platforms that support Next.js. The background job endpoint must remain accessible so that Inngest can trigger the BuildrAgent function. The sandbox template must also be available so generated websites can run correctly.

## 5.7 Testing Observations

Testing showed that the main challenge in the project is coordinating multiple services. A generation request depends on authentication, database writes, background events, AI model response and sandbox availability.

The use of clear error messages is important because external services may fail or take time. If the AI does not produce files, the system stores an error assistant message so the user receives feedback instead of a blank result.

Another observation is that the project workspace should remain responsive while generation is running. Background execution supports this requirement because the frontend is not forced to wait for the entire AI workflow inside a single request.



## **Chapter 6 : Conclusion and Future Scope**

This chapter summarises the project outcome and presents the future scope of Buildr.

## **Conclusion**

Buildr successfully implements an AI-powered website builder that converts prompts into live web previews. It combines a modern frontend, protected server procedures, database persistence, background AI execution and sandboxed code running.

The project fulfils its objective of reducing the effort required for web prototyping. It also provides educational value because generated code can be inspected directly from the browser.

The system shows that AI becomes more powerful when it is integrated with reliable engineering components. Authentication protects user data, Prisma stores project history, Inngest manages long tasks and E2B provides a safe execution environment.

**Benefits:**

Buildr offers benefits to both technical and non-technical users. It saves time, supports experimentation and gives quick visual output from written requirements.

- Fast conversion of ideas into website prototypes.
- Project history and iterative prompting.
- Safe sandbox execution of generated code.
- Transparent code viewer for learning and review.
- Modern and maintainable full-stack architecture.
- Useful academic demonstration of AI-assisted software development.

## **Future Work**

Future improvements can include GitHub export, direct deployment, version history, team collaboration, better generated-code testing, custom design controls, file download, reusable templates and stronger monitoring of AI output quality.

The project can also be extended with multi-page application generation, database-backed generated apps and automatic documentation for each generated website. Another useful improvement would be image upload support so that users can provide visual references along with prompts.

A future version may also include an admin dashboard for monitoring usage, generation failures and sandbox resource consumption.

## **References**

The following references were used for project understanding and report preparation.

### **Books**

1. Software Engineering by Roger S. Pressman.
2. Database System Concepts by Abraham Silberschatz, Henry F. Korth and S. Sudarshan.
3. Web Engineering: A Practitioner's Approach by Roger S. Pressman and David Lowe.

### **Articles/Journals**

1. Articles related to AI-assisted software development and prompt-based code generation.
2. Technical articles on full-stack TypeScript application architecture.
3. Documentation notes prepared from the Buildr project source code.

### **Websites**

1. <https://nextjs.org/>
2. <https://react.dev/>
3. <https://www.prisma.io/>
4. <https://clerk.com/>
5. <https://www.inngest.com/>
6. <https://openai.com/>
7. <https://e2b.dev/>